

TUGATAI AI SAAS PLATFORM

PRODUCTION SYSTEM ARCHITECTURE DOCUMENT

Prepared By

Md. Shahriar Hossain Jihad

Version

Version 1.0

Architecture Type

Modular Monolithic SaaS Architecture

Date

May 2026

TABLE OF CONTENTS

1. Architecture Overview
2. System Goals
3. Architectural Principles
4. High-Level Architecture
5. Frontend Architecture
6. Backend Architecture
7. AI Architecture
8. Database Architecture
9. Realtime Architecture
10. Queue Architecture
11. Workflow Engine Architecture
12. CRM Architecture
13. Omnichannel Architecture
14. Security Architecture

15. SaaS Multi-Tenant Architecture
 16. Branch Management Architecture
 17. Deployment Architecture
 18. Scalability Strategy
 19. Folder Structure
 20. Event Flow Architecture
 21. Infrastructure Recommendations
 22. Future Microservice Extraction Strategy
 23. Conclusion
-

1. ARCHITECTURE OVERVIEW

Tugatai AI is designed as a scalable modular monolithic SaaS platform focused on:

- omnichannel customer engagement
- AI-assisted business automation
- CRM workflows
- booking workflows
- campaign automation
- branch-based operations
- multi-tenant SaaS scalability

The architecture prioritizes:

- scalability
 - maintainability
 - clean modularity
 - future microservice migration
 - realtime communication
 - AI orchestration
 - operational efficiency
-

2. SYSTEM GOALS

The architecture is designed to achieve:

- centralized omnichannel communication
- AI-assisted customer engagement
- scalable SaaS tenancy
- branch-level operational management
- deterministic business logic
- high realtime responsiveness
- future extensibility

- simplified deployment
- maintainable codebase

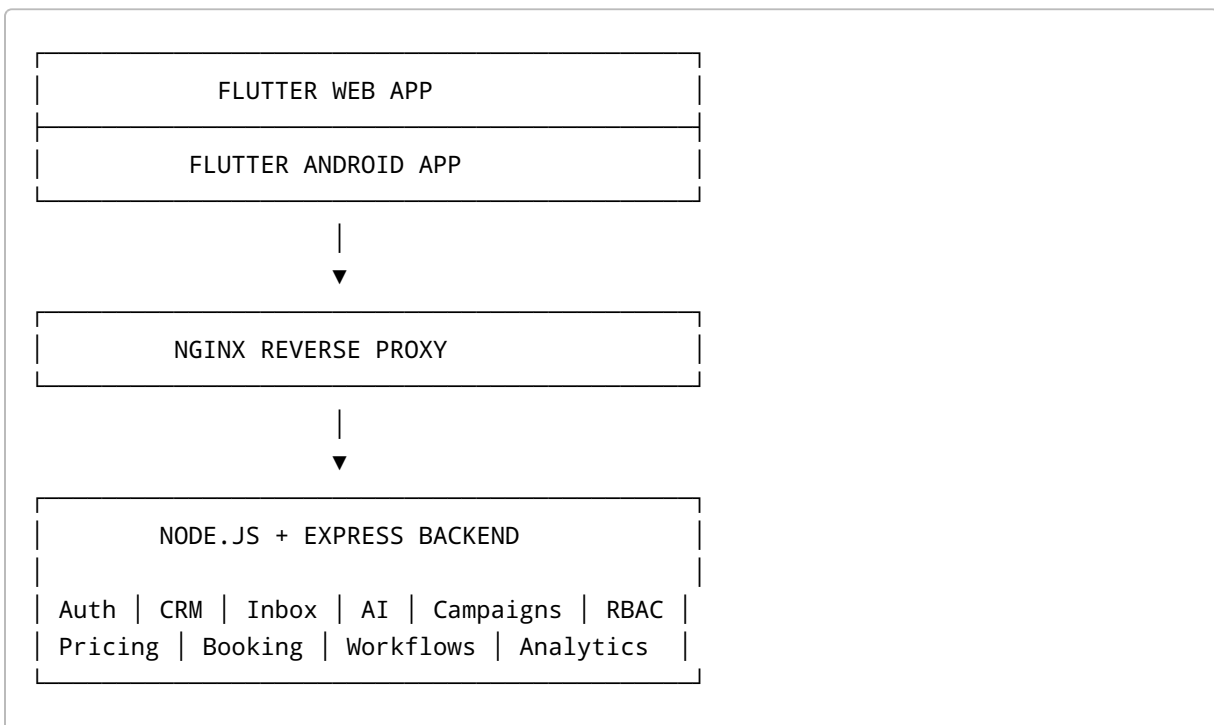
3. ARCHITECTURAL PRINCIPLES

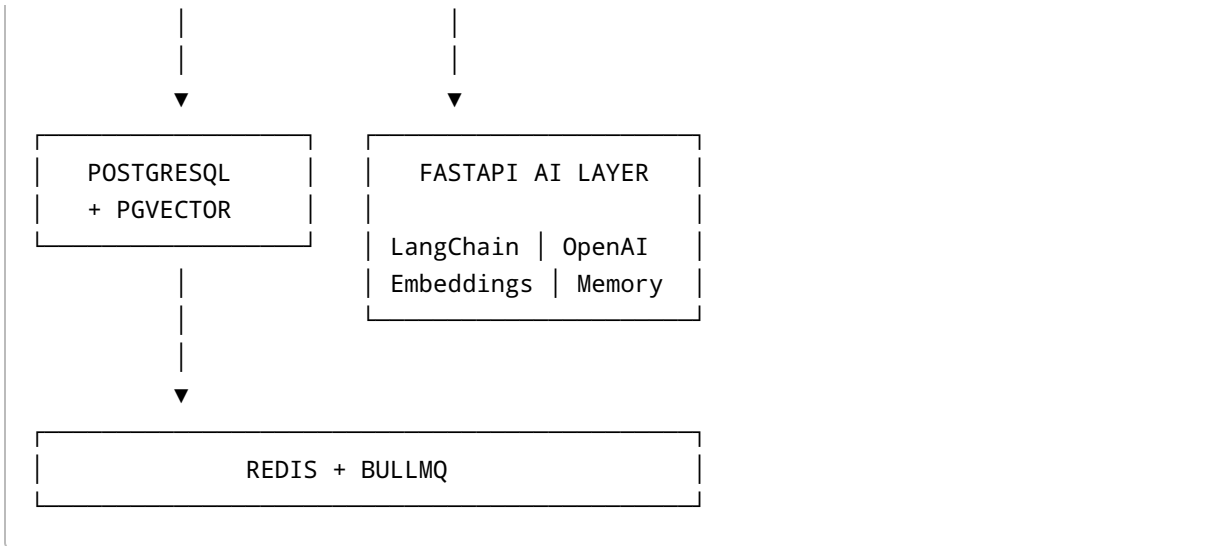
The platform follows:

- Modular Monolithic Architecture
- MVC Architecture
- Repository Pattern
- DRY Principle
- KISS Principle
- YAGNI Principle
- Event-Driven Processing
- Clean Separation of Concerns
- Tenant Isolation
- Branch Isolation
- Scalable Modular Design

4. HIGH-LEVEL ARCHITECTURE

System Architecture Diagram





5. FRONTEND ARCHITECTURE

Frontend Technology Stack

Component	Technology
Web Frontend	Flutter Web
Mobile App	Flutter Android
Realtime	Socket.IO
Authentication	JWT

Frontend Modules

```
Frontend Modules
|
├─ Authentication
├─ Dashboard
├─ Omnichannel Inbox
├─ CRM
├─ Analytics
├─ Campaign Management
├─ AI Training
├─ Knowledge Base
├─ Billing & Subscription
```

- └─ Workflow Management
- └─ Branch Management
- └─ Tenant Settings
- └─ Mobile Operations

Frontend Responsibilities

The frontend layer handles:

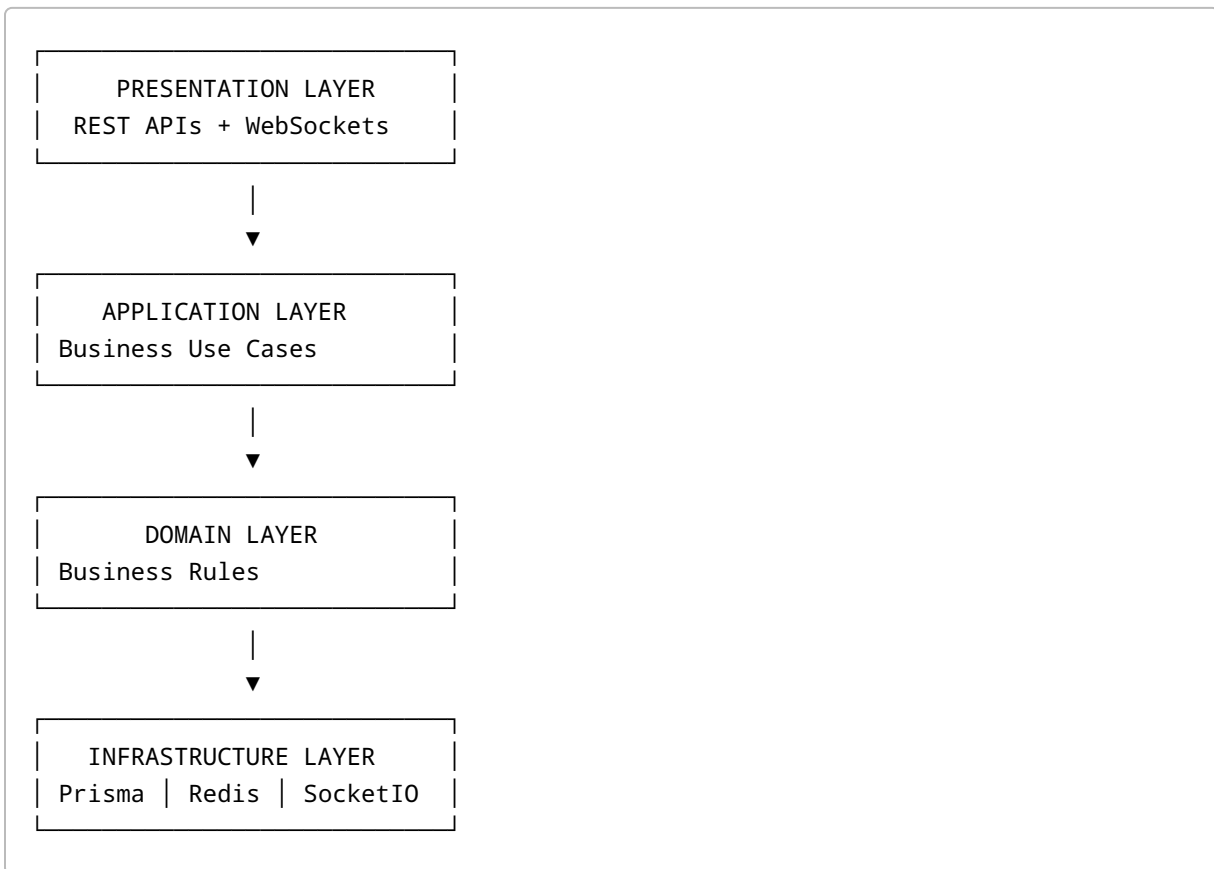
- user interactions
- realtime UI updates
- dashboard rendering
- conversation rendering
- CRM interfaces
- analytics visualization
- campaign interfaces
- authentication state
- mobile operations

6. BACKEND ARCHITECTURE

Backend Technology Stack

Component	Technology
Runtime	Node.js
Framework	Express.js
ORM	Prisma
Architecture	Modular Monolith
Realtime	Socket.IO
Queue	BullMQ
Cache	Redis

Backend Layered Architecture



Backend Module Architecture

```
src/  
|  
├── modules/  
│   ├── auth/  
│   ├── tenant/  
│   ├── branch/  
│   ├── user/  
│   ├── inbox/  
│   ├── crm/  
│   ├── ai/  
│   ├── campaign/  
│   ├── booking/  
│   ├── pricing/  
│   ├── workflow/  
│   └── analytics/
```

```
| | | billing/
| | | notifications/
| | | knowledge-base/
| |
| | shared/
| | infrastructure/
| | config/
| | prisma/
| | main/
```

7. AI ARCHITECTURE

AI Technology Stack

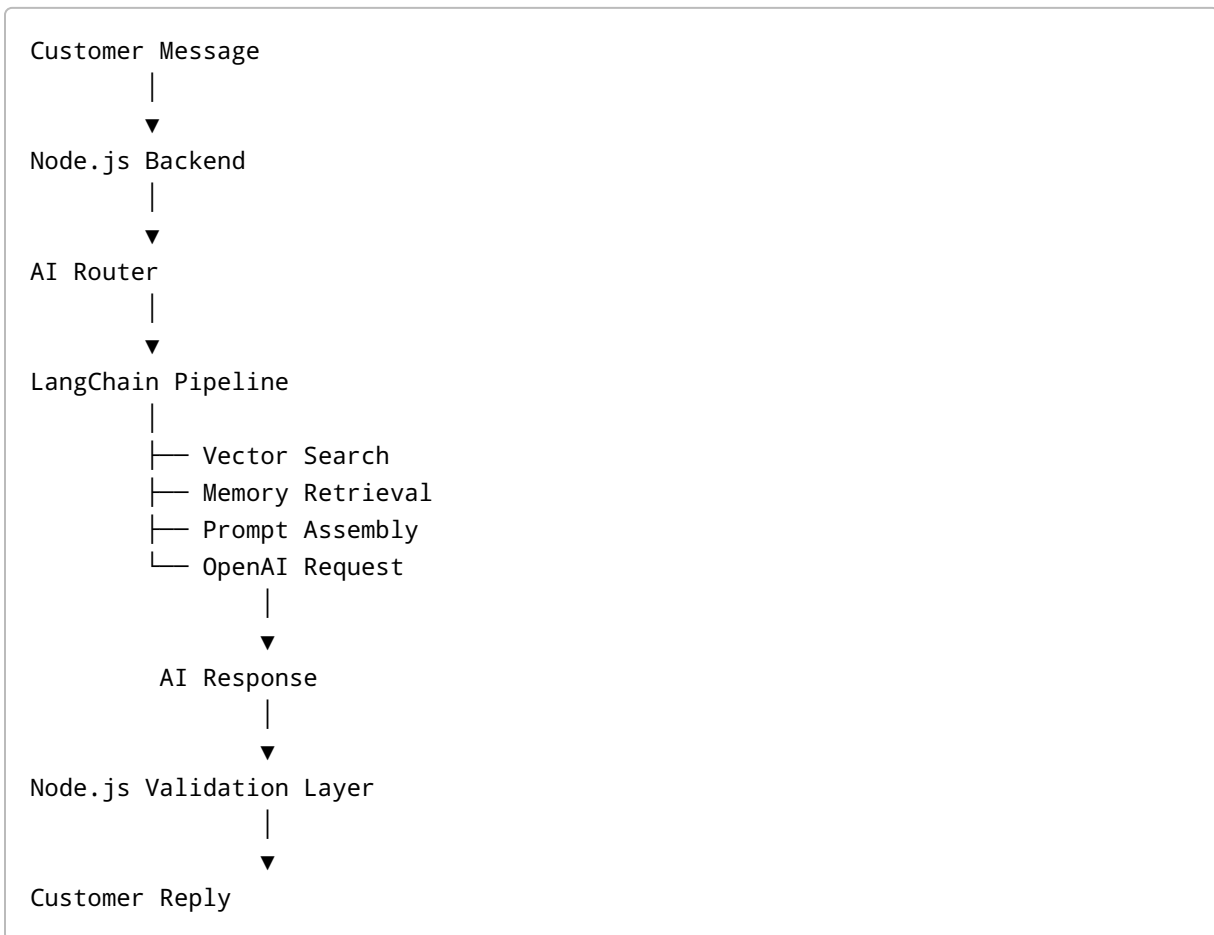
Component	Technology
AI Framework	FastAPI
LLM	OpenAI
Future LLM	Anthropic
RAG Framework	LangChain
Embeddings	pgvector
Memory	Vector Memory

AI Service Responsibilities

The AI layer handles:

- intent detection
- entity extraction
- RAG retrieval
- response generation
- summarization
- AI memory
- sentiment analysis
- multilingual response generation

AI Architecture Flow



Important AI Rule

AI MUST NOT:

- directly calculate pricing
- directly modify bookings
- directly mutate CRM state
- bypass backend validation

The backend remains the source of truth.

8. DATABASE ARCHITECTURE

Database Technology

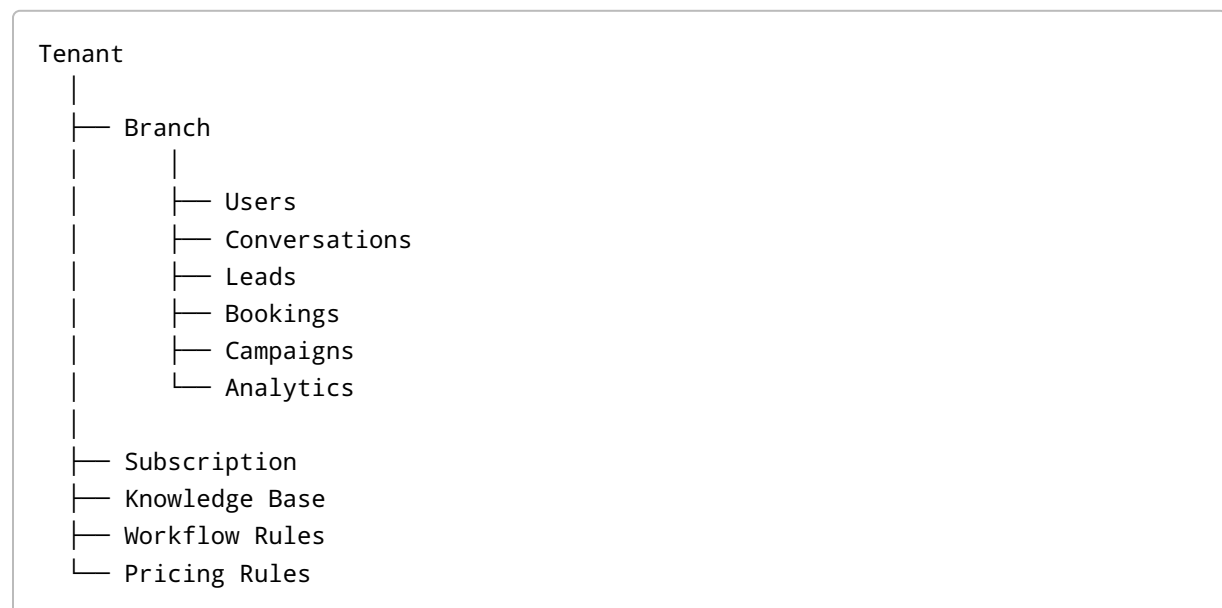
Component	Technology
Main Database	PostgreSQL
Vector Search	pgvector
ORM	Prisma

Database Design Strategy

The system uses:

- shared database architecture
- tenant_id isolation
- branch hierarchy isolation
- relational data modeling
- audit-based tracking

High-Level Database ERD



Core Tables

```
tenants
branches
users
roles
permissions
channels
conversations
messages
contacts
leads
crm_notes
bookings
pricing_rules
campaigns
workflow_rules
subscriptions
payments
knowledge_documents
ai_embeddings
analytics
audit_logs
notifications
```

9. REALTIME ARCHITECTURE

Realtime Technology

Component	Technology
WebSocket Layer	Socket.IO
Pub/Sub	Redis

Realtime Communication Flow

```
Meta Webhooks
  |
  ▼
Express Event Handler
```



Realtime Features

- live messaging
- typing indicators
- conversation updates
- CRM updates
- notifications
- campaign updates
- analytics refresh

10. QUEUE ARCHITECTURE

Queue Technology

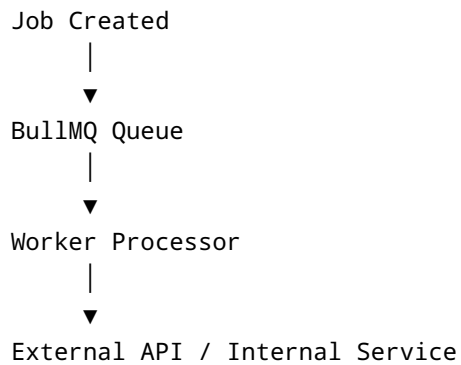
Component	Technology
Queue Engine	BullMQ
Queue Storage	Redis

Queue Responsibilities

The queue system handles:

- campaign processing
- AI jobs
- retry jobs
- scheduled workflows
- delayed follow-ups
- notifications
- analytics aggregation

Queue Processing Flow

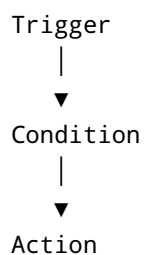


11. WORKFLOW ENGINE ARCHITECTURE

Workflow Engine Type

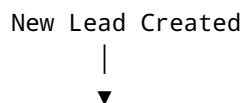
Rule-Based Workflow Engine

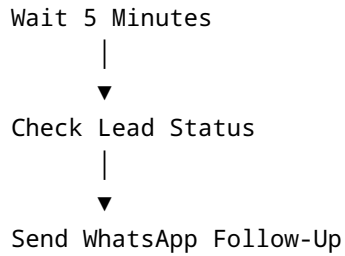
Workflow Structure



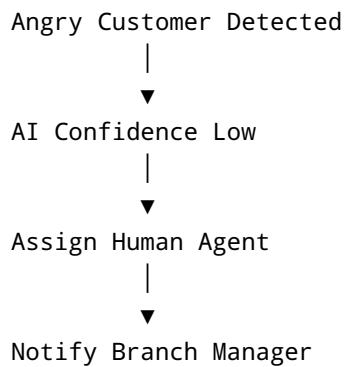
Workflow Examples

Lead Follow-Up Workflow





Escalation Workflow



Workflow Responsibilities

The workflow engine handles:

- lead follow-ups
- booking automation
- escalation automation
- campaign scheduling
- reminders
- operational automation

12. CRM ARCHITECTURE

CRM Goals

The CRM system is designed to:

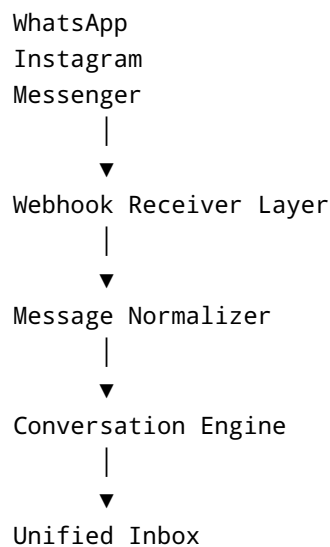
- track leads

13. OMNICHANNEL ARCHITECTURE

Supported Channels

- WhatsApp Cloud API
 - Instagram Messaging API
 - Facebook Messenger API
-

Omnichannel Flow



Omnichannel Responsibilities

The channel layer handles:

- webhook ingestion
 - message normalization
 - media handling
 - outbound dispatching
 - channel synchronization
-

14. SECURITY ARCHITECTURE

Security Layers

```
graph TD; A[Authentication] --> B[Authorization]; B --> C[Tenant Isolation]; C --> D[Branch Isolation]; D --> E[Request Validation]; E --> F[Rate Limiting];
```

Authentication
|
Authorization
|
Tenant Isolation
|
Branch Isolation
|
Request Validation
|
Rate Limiting

Authentication

Technology:

- JWT
- Refresh Tokens

Authorization

Role-Based Access Control (RBAC)

Roles:

- Super Admin
- Tenant Owner
- Branch Manager
- Agent

Sensitive Data Security

Protected secrets include:

- OpenAI keys

- Meta API keys
 - Stripe keys
 - Database credentials
 - JWT secrets
-

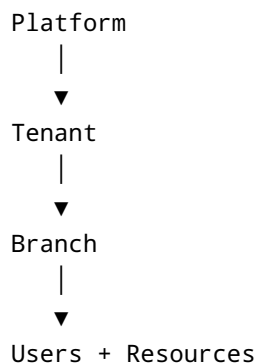
15. SAAS MULTI-TENANT ARCHITECTURE

Multi-Tenant Strategy

The platform uses:

- shared database architecture
 - tenant_id isolation
 - scoped queries
 - tenant-aware middleware
-

Tenant Hierarchy



Tenant Isolation Rules

Each tenant has isolated:

- conversations
- CRM
- analytics
- campaigns
- workflows
- pricing rules
- AI knowledge

- bookings

16. BRANCH MANAGEMENT ARCHITECTURE

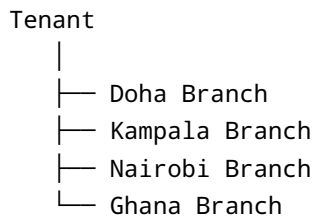
Branch Structure

One tenant can contain multiple branches.

Each branch can have:

- separate agents
- separate inboxes
- separate campaigns
- separate analytics
- separate workflows
- separate CRM visibility

Branch Architecture Flow



Branch-Level Permissions

Branch managers can only access:

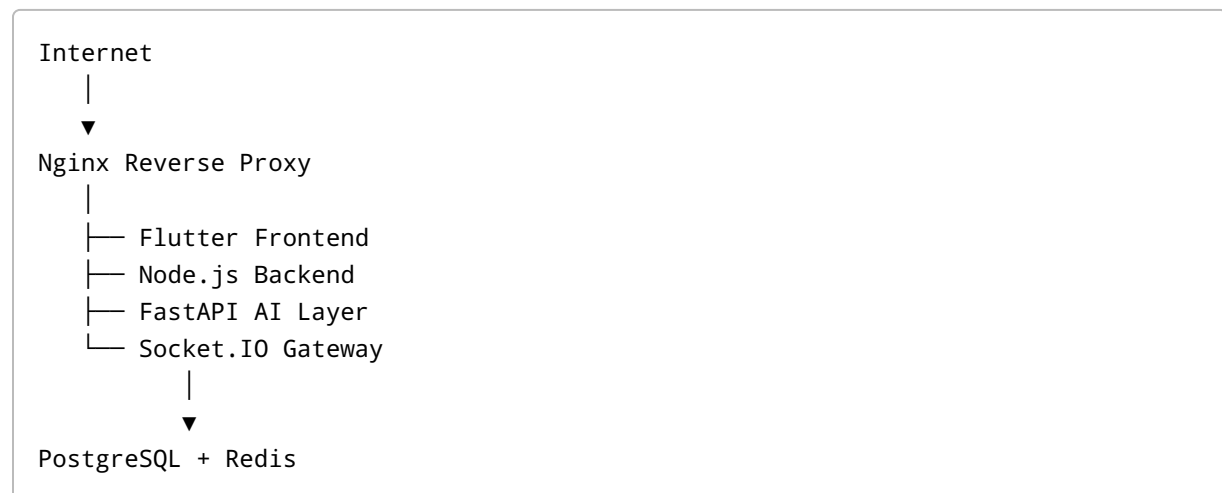
- assigned conversations
- assigned agents
- assigned CRM records
- assigned analytics

17. DEPLOYMENT ARCHITECTURE

Deployment Infrastructure

Component	Technology
Reverse Proxy	Nginx
Containerization	Docker
Hosting	VPS
SSL	Let's Encrypt

Deployment Diagram



File Storage Strategy

MVP

- Multer + local storage

Future

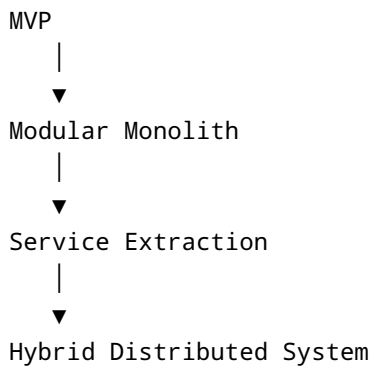
- AWS S3

18. SCALABILITY STRATEGY

Current Architecture

Modular Monolith

Future Scalability Path



Future Extraction Candidates

- AI Service
 - Campaign Engine
 - Workflow Engine
 - Analytics Service
 - Notification Service
-

19. FOLDER STRUCTURE

Backend Folder Structure

```
src/  
|  
├── modules/  
|   ├── auth/  
|   ├── tenant/  
|   ├── branch/  
|   └── inbox/
```


21. INFRASTRUCTURE RECOMMENDATIONS

Recommended Production Stack

Layer	Recommendation
Frontend	Flutter Web
Mobile	Flutter Android
Backend	Node.js + Express
AI	FastAPI
Database	PostgreSQL
Vector Search	pgvector
Cache	Redis
Queue	BullMQ
Storage	Local → S3
Deployment	Docker + VPS

22. FUTURE MICROSERVICE EXTRACTION STRATEGY

Extraction Strategy

The architecture is intentionally designed for future service extraction.

Extraction Candidates

AI Service

Can become independent due to:

- heavy compute
- async processing

- vector workloads
-

Campaign Engine

Can become independent due to:

- scheduling
 - high throughput
 - distributed workers
-

Analytics Service

Can become independent due to:

- aggregation workloads
 - reporting jobs
 - background processing
-

Migration Philosophy

The system follows:

- modular boundaries
- service-oriented module design
- clean interfaces
- repository abstraction

This minimizes future migration complexity.

23. CONCLUSION

Tugatai AI SaaS Platform is designed as a scalable modular omnichannel AI business operating system.

The architecture combines:

- omnichannel communication
- AI orchestration
- CRM
- workflow automation
- booking systems
- campaign management

- branch operations
- SaaS scalability

into a unified production-ready platform.

The architecture prioritizes:

- scalability
- maintainability
- operational efficiency
- clean modularity
- realtime performance
- future extensibility
- production readiness

The platform is structured to support both:

- rapid MVP development
- long-term enterprise scalability